

飞机票订票系统

数据库课程设计报告

课程：数据库原理与应用

项目技术栈：Spring Boot 3.2 · MyBatis · Flyway · MySQL 8 · Vue 3

报告重点：数据库规划、概念设计、逻辑设计与物理实现

摘要

本报告以 AirTicket 飞机票订票系统为对象，围绕在线机票销售中航班检索、三舱余票控制、订单支付、核销、退订、改签和实时轨迹等业务，完成从需求分析到物理实现的数据库设计。系统采用 MySQL 8 与 InnoDB，采用 9 张核心表保存用户、航空公司、机场、机型、航班、订单、渠道、轨迹和常乘机人数据。设计的重点是把航班座位、订单状态和资金状态放在可验证的关系模式中，并以事务与行级锁保证高并发订票不发生超卖。实际项目通过 Flyway 管理 V1 至 V6 的迁移，内置 42 条航班、66 条订单、12 个用户等演示数据，约含 175 条实体记录和 374 条关联关系。报告同时说明逻辑外键关系与当前物理 DDL 的差异：业务关系均在模型中明确，已部署的 V1 建表脚本主要以唯一索引、应用层校验和删除引用保护维持引用完整性。

关键词：飞机票订票；MySQL；ER 模型；第三范式；InnoDB；事务并发控制

第一章 数据库规划

1.1 系统开发背景与意义

民航客票业务具有航班时效性强、库存有限、销售渠道多和退款改签频繁的特点。传统人工查询和分散登记难以同步各渠道库存，容易出现信息滞后、重复销售和售后记录不完整等问题。在线订票系统将航班、机场、机型、订单及支付信息集中管理，使旅客可以按出发地、到达地、日期和价格检索航班，并完成选舱、下单、支付和订单追踪。数据库是这一流程的事实来源：它既保存基础数据，也记录每一次库存与订单状态变化。

本课题的价值在于把典型业务规则转化为数据约束与访问路径。例如，一趟航班由航班号和出发时间共同唯一确定；同一用户不得重复维护同一常乘机人；订单只对应一名下单用户、一趟航班和一个渠道；航班的出发与到达机场是同一实体的两个不同角色。通过这些设计，系统能够在可追溯、可统计的前提下支撑线上售票。

1.2 系统可行性分析

维度	分析结论
技术可行性	Spring Boot 3.2、MyBatis 与 MySQL 8 组合成熟；Flyway 可把结构和种子数据作为版本化迁移执行；Vue 3 前端通过 REST 接口访问后端。
经济可行性	采用开源框架和 Docker 交付，开发、部署与维护成本较低；MySQL 能满足课程设计规模。
操作可行性	旅客自助注册、搜索和支付；商家负责放票、核销和改签；管理员维护用户与基础数据，职责清晰。

1.3 系统总体结构

系统采用前后端分离结构。Vue 3 前端提供登录、航班搜索、选舱、订单中心、个人中心和管理后台；Spring Boot 后端按 interfaces → application → domain → infrastructure 分层处理接口、应用编排、领域规则和持久化；MyBatis 位于基础设施层执行 SQL；MySQL 8 持久化 9 张核心表；Flyway 按版本创建和演进模式。管理员由初始化端点创建并分配商家，旅客自行注册。JWT 负责无状态身份识别，角色注解负责权限校验。

第二章 系统定义与需求分析

2.1 系统定义

AirTicket 是一个面向管理员、商家和旅客的在线飞机票订票系统：以航班库存为核心，支持三舱定价、订票支付、核销、退订退款、改签、实时轨迹和经营统计。

角色	主要职责	数据访问范围
管理员	初始化账号、用户管理、航司/机场/机型/渠道 CRUD、放票管理与统计	全局基础数据、用户、航班和订单
商家	维护其放出的航班，办理核销和改签	受 created_by 归属控制的航班与相关业务数据
旅客	注册登录、检索、选舱下单、支付、退订、查看轨迹和维护常乘机人	仅本人资料、本人订单与常乘机人

2.2 功能需求分析

功能	业务说明	是否写库
登录注册	校验账号并签发 JWT；旅客注册写入 sys_user。	是
航班搜索	按地区、日期、价格区间检索，展示三舱价与余票。	否
订票	按选定舱级锁定航班、扣减对应余票、生成 orders。	是
支付	两段式模拟支付：支付中后确认或回调，更新订单状态。	是
核销	商家或管理员将订单置为已核销。	是
退订退款	计算退款，回补余票，更新订单支付/业务状态。	是
改签	订单改指向新航班，同舱互换库存并计算差额。	是
实时轨迹	查询 route 中某航班当前位置、速度与剩余信息。	可写可读
个人中心	维护资料、密码和常乘机人。	是
管理后台	维护用户及航司、机场、机型、渠道、航班基础数据。	是
统计	热门航班、营收总览和旅客消费排行。	否

2.3 数据需求分析

从上述功能反推，身份与权限需要 sys_user；航司、机场、机型构成航班基础资源；flight 保存一次具体飞行的时刻、路线、三舱库存、价格、退票费、登机口和状态；orders 将旅客、航班和渠道连接起来，并记录支付、出票、取消时间与金额；route 保存每趟航班当前的实时状态；passenger 用两条指向 sys_user 的引用表达“某用户的常乘机人”。channel 则保留第三方销售或支付渠道的网关地址。

2.4 性能需求

航班搜索、我的订单和统计是主要读路径。搜索应利用 flight(region_dep, region_arr, datetime_dep) 缩小时间范围；精确航班使用 code 与 datetime_dep 的组合唯一索引；订单列表先按 id_user 过滤；取消航班时按

id_flight 找订单。订票、退票和改签是关键写路径，必须在同一事务中使用 SELECT ... FOR UPDATE 锁住涉及的 flight 行，检查余票后再更新座位和订单，以行锁序列化同航班同舱的竞争。

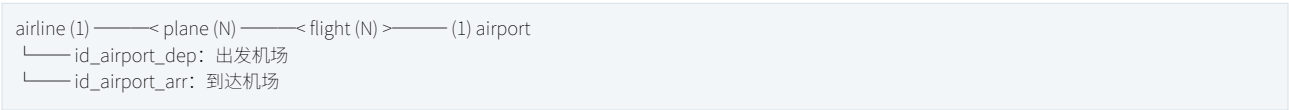
2.5 安全需求

密码使用 BCrypt 摘要，JWT 仅承载身份上下文而不保存会话状态；@RequireRole 与拦截器校验角色；订单查询必须把当前用户 id 作为条件，防止越权读取；基础数据删除前检查引用关系；数据库账户按应用读写、迁移、备份等职责分离。对输入的邮箱、手机号、经纬度及飞机重量等数据，使用 CHECK 或应用层校验共同限制非法值。

第三章 数据库概念设计

3.1 局部 ER 图与联系说明

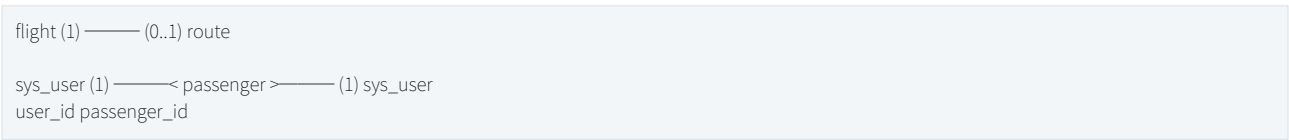
ER-1: 航司 - 机型 - 航班 - 机场。实体包括 airline、plane、flight、airport。一个航空公司可拥有多种机型，每个机型必须隶属一个航司，故 airline 1:N plane, plane 对该联系全参与；一个机型可执行多趟航班，而一趟航班使用一个机型，故 plane 1:N flight, flight 全参与。flight 与 airport 存在两条角色不同的 N:1 联系：出发机场 id_airport_dep 与到达机场 id_airport_arr。一个机场可以是多趟航班的出发地或到达地，航班必须同时具有两个机场角色。



ER-2: 订单 - 用户 - 渠道 - 航班。sys_user 与 orders 为 1:N，一名用户可以没有订单也可以有多笔订单，订单必须由一名用户创建；channel 与 orders 为 1:N，订单必须选择一个渠道；flight 与 orders 为 1:N，一趟航班可以产生多笔订单，订单必须指定一趟航班。orders 是交易事实实体，属性包括订单号、舱级、金额、支付状态、订单状态以及支付/出票/取消时间。



ER-3: 轨迹 - 常乘机人。route 与 flight 当前按 1:1 设计，route.id_flight 的唯一性使一趟航班至多保留一条实时状态记录；航班可以暂时无轨迹，route 一旦存在必须属于航班。passenger 是 sys_user 的自关联：user_id 表示列表归属人，passenger_id 表示被保存的常乘机人，两者均指向 sys_user.id。一个用户可维护多位常乘机人，也可被多个用户保存。



3.2 实体属性、主键和参与约束

实体	主键及代表性属性	参与约束
sys_user	<u>id</u> , username, password, role, status	订单和常乘机人联系均可部分参与
airline	<u>id</u> , name	可暂时没有机型
airport	<u>id</u> , name, region	可暂时没有相关航班
plane	<u>id</u> , model_name, 三舱最大座位数	必须属于一个航司；可暂时无航班
flight	<u>id</u> , code, datetime_dep, 三舱余票与价格	必须使用一个机型及两个机场角色
orders	<u>id</u> , code, cabin_class, 金额和状态	必须关联用户、航班和渠道
route	<u>id</u> , 经纬度、高度、速度、时间戳	必须关联航班
passenger	<u>id</u> , user_id, passenger_id	两个参与端均为用户
channel	<u>id</u> , channel_name, api_gateway_url	可暂时没有订单

3.3 全局 ER 图合并原则

合并局部 ER 图时以业务语义和标识符消除冲突：三个局部图中的 flight、sys_user、airport 等同名实体只保留一个，统一使用 id

作为标识；航班与机场的两个联系不合并为一个无角色联系，而是保留“出发”和“到达”角色；orders 连接交易相关实体，不把用户、航司名称等可由关联得到的属性复制进订单。由此得到 9 个实体及其完整联系网络。

第四章 数据库逻辑设计

4.1 ER 图转关系模式

实体转化为关系表；1:N 联系在 N 端加入外键列；1:1 联系将 flight 的标识放在 route 中并施加唯一性；sys_user 的自关联通过 passenger 关系实现。下列下划线字段为主键，箭头表示逻辑外键指向。实际 V1 迁移脚本已创建表与唯一/普通索引；这些箭头描述的是概念和逻辑引用关系，当前物理脚本未显式声明 FOREIGN KEY。

关系模式	说明
sys_user(<u>id</u> , username, password, real_name, age, email, phone, status, role, create_at)	用户与角色信息
airline(<u>id</u> , name, create_at)	航空公司
airport(<u>id</u> , name, region, create_at)	机场与地区
plane(<u>id</u> , id_airline→airline.id, model_name, length, wingspan, height, max_takeoff_weight_kg, max_landing_weight_kg, max_seat_first_class, max_seat_business_class, max_seat_economy_class, create_at)	机型参数
channel(<u>id</u> , channel_name, api_gateway_url)	销售/支付渠道
flight(<u>id</u> , id_plane→plane.id, id_airport_dep→airport.id, id_airport_arr→airport.id, code, datetime_dep, datetime_arr, region_dep, region_arr, distance, seat_first_class, seat_business_class, seat_economy_class, price, price_business_class, price_first_class, cancellation_fee, gate, status, created_by, create_at)	一次具体飞行
orders(<u>id</u> , id_flight→flight.id, id_user→sys_user.id, id_channel→channel.id, code, cabin_class, total_price, total_tax, pay_status, order_status, pay_time, issue_time, cancel_time, remark, create_at)	交易订单
route(<u>id</u> , id_flight→flight.id, distance_remain, time_remain, altitude, speed, latitude, longitude, time_stamp, create_at)	当前实时轨迹
passenger(<u>id</u> , user_id→sys_user.id, passenger_id→sys_user.id, create_at)	用户常乘机人自关联

4.2 规范化分析

所有关系均以单一代理主键 id 为候选键，主要非主属性完全依赖于 id，因此不存在复合主键的部分依赖，满足第二范式。sys_user 中 username 是唯一业务标识，但姓名、联系方式、角色和状态均描述该用户本身；airline、airport、channel 的名称或网关均由各自 id 决定；plane 的尺寸、重量与座位容量由机型 id 决定；route 的位置与状态由轨迹 id 决定。上述表不存在非主属性决定另一非主属性的必然函数依赖。

flight 中出发/到达地区、时刻、距离、库存、票价和状态都由具体航班 id 决定；orders 中订单号、舱级、金额和各状态时间由订单 id 决定。订单仅保存 total_price 和 total_tax，而不冗余保存航司名称、机场名称、机型参数等可经 id_flight 联接得到的数据，避免因基础数据更新引发大量订单记录修改。订单金额是成交时点的业务快照，独立于以后航班价格的调整，符合交易留痕需要。因而不是对航司等维度信息的冗余复制。

flight 中 region_dep 与 region_arr 可由机场表的 region 推导，严格说是有意的受控冗余。保留它们可以让“地区 - 地区 - 日期”检索直接走联合索引，避免搜索高峰频繁联接 airport；代价是机场地区更新时需同步相关航班。项目将该同步限定在基础数据维护流程中，并由应用层校验和迁移管理控制，因此在性能与一致性之间作出可接受的折中。除这两个检索冗余字段外，关系模式满足第三范式的设计目标。

4.3 核心建表 SQL

```
CREATE TABLE sys_user (  
id BIGINT PRIMARY KEY AUTO_INCREMENT, username VARCHAR(50) NOT NULL,  
password VARCHAR(60) NOT NULL, real_name VARCHAR(20), age TINYINT UNSIGNED,  
email VARCHAR(50), phone VARCHAR(20), status BOOLEAN DEFAULT TRUE,  
role VARCHAR(30) NOT NULL, create_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
UNIQUE INDEX idx_username (username),  
CHECK (email IS NULL OR email REGEXP '^([A-Za-z0-9._%+]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,})$'),  
CHECK (phone IS NULL OR phone REGEXP '^1[3-9][0-9]{9}$'),  
CHECK (email IS NOT NULL OR phone IS NOT NULL)  
);  
  
CREATE TABLE flight (  
id BIGINT PRIMARY KEY AUTO_INCREMENT, id_plane BIGINT NOT NULL,  
id_airport_dep BIGINT NOT NULL, id_airport_arr BIGINT NOT NULL, code VARCHAR(10) NOT NULL,  
datetime_dep DATETIME NOT NULL, datetime_arr DATETIME NOT NULL,  
region_dep VARCHAR(20) NOT NULL, region_arr VARCHAR(20) NOT NULL, distance INT UNSIGNED NOT NULL,  
seat_first_class SMALLINT UNSIGNED NOT NULL, seat_business_class SMALLINT UNSIGNED NOT NULL,  
seat_economy_class SMALLINT UNSIGNED NOT NULL, price DECIMAL(10,2) UNSIGNED NOT NULL,  
cancellation_fee DECIMAL(10,2) UNSIGNED NOT NULL, gate VARCHAR(10), status VARCHAR(30) NOT NULL,  
create_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, UNIQUE INDEX idx_feature (code, datetime_dep)  
);  
-- V3 增量： price_business_class 与 price_first_class； V4 增量： created_by。  
  
CREATE TABLE orders (  
id BIGINT PRIMARY KEY AUTO_INCREMENT, id_flight BIGINT NOT NULL, id_user BIGINT NOT NULL,  
id_channel BIGINT NOT NULL, code VARCHAR(50) NOT NULL, total_price DECIMAL(10,2) NOT NULL,  
total_tax DECIMAL(10,2) NOT NULL, pay_status VARCHAR(30) NOT NULL, order_status VARCHAR(30) NOT NULL,  
pay_time DATETIME, issue_time DATETIME, cancel_time DATETIME, remark VARCHAR(100),  
create_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, UNIQUE INDEX idx_code (code)  
);  
-- V3 增量： cabin_class VARCHAR(20) NOT NULL DEFAULT 'ECONOMY'。
```


第五章 数据库物理设计

5.1 存储引擎与字符集

核心表采用 InnoDB。其事务、行级锁和崩溃恢复能力适合订票、退票与改签等需原子执行的交易。字符集使用 utf8mb4，以完整支持中文名称、备注和可能出现的 emoji。虽然 V1 脚本未逐表指定引擎与字符集，部署环境应统一以 MySQL 的 InnoDB 和 utf8mb4 默认配置创建，或在生产脚本中显式声明。

5.2 数据库物理实现清单

表名	引擎/字符集	关键索引	用途
sys_user	InnoDB / utf8mb4	idx_username 唯一	登录与身份识别
airline	InnoDB / utf8mb4	idx_airline_name 唯一	航司基础数据
airport	InnoDB / utf8mb4	idx_airport_name 唯一；idx_airport_region	地区搜索
plane	InnoDB / utf8mb4	idx_model_name 唯一	机型与舱位容量
channel	InnoDB / utf8mb4	idx_channel_name 唯一	渠道识别
flight	InnoDB / utf8mb4	idx_feature；地区日期联合索引	航班检索与库存
orders	InnoDB / utf8mb4	idx_code；id_user；idx_orders_flight	订单查询与取消
route	InnoDB / utf8mb4	idx_route_flight 唯一	实时轨迹
passenger	InnoDB / utf8mb4	idx_passenger(user_id, passenger_id)	常乘机人去重

5.3 索引设计

关键索引包括：flight(code, datetime_dep) 唯一索引 idx_feature，保障具体航班唯一；flight(region_dep, region_arr, datetime_dep) 服务航班搜索；orders(id_user) 服务“我的订单”；orders(id_flight) 服务航班取消时定位订单；orders(code) 唯一保证订单号；route(id_flight) 唯一保证一航班一条当前轨迹；sys_user(username) 唯一服务登录；airport(region) 服务地区筛选。项目迁移已实现其中的 idx_feature、idx_code、idx_route_flight、idx_username、idx_airport_region 和 idx_orders_flight；下列是搜索联合索引的建表示例。

```
CREATE INDEX idx_flight_search ON flight (region_dep, region_arr, datetime_dep);
```

5.4 事务与并发控制

订票操作必须先开启事务，再按航班主键读取并加排他锁。应用根据 cabin_class 判断相应的余票字段，只有余票大于 0 才更新库存并插入订单；任一语句失败均回滚。退订和改签同样锁定涉及的航班，改签时按固定的 id 升序加锁两趟航班，降低死锁机会。InnoDB 的行锁使不同航班可以并行售卖，而同一航班的冲突订票被串行化。

```
START TRANSACTION;
SELECT id, seat_economy_class, seat_business_class, seat_first_class
FROM flight WHERE id = ? FOR UPDATE;
-- 校验所选舱余票 > 0 后：更新对应 seat_*_class，并 INSERT INTO orders (...)
COMMIT;
```

5.5 数据安全与备份

权限方面，将迁移账户、业务应用账户和备份账户分离；应用端只授予必要的 DML 权限。结构变化通过 Flyway V1 至 V6 的版本化迁移交付，避免人工环境漂移。备份策略采用每日逻辑备份与定期全量备份，保留可恢复窗口；上线前验证备份文件，并进行恢复演练。对订单等关键表还应记录迁移版本和恢复时间，保证故障后可追溯。

第六章 系统实现与关键 SQL

6.1 技术栈与分层架构

后端基于 Spring Boot 3.2 与 JDK 17，使用 MyBatis 映射 SQL、Flyway 管理迁移、MySQL 8 存储数据；前端为 Vue 3 + Vite。DDD 分层中，interfaces 负责 HTTP 接口、认证与参数接收；application 编排订票、支付、退订和改签用例；domain 表达库存、订单状态等业务规则；infrastructure 中的 Repository/Mapper 执行 SQL。SQL 不散落于控制器或领域对象，从而使权限与事务边界更清晰。

6.2 关键功能 SQL

(1) 航班搜索。按条件拼接动态谓词，地区和时间条件与联合索引顺序一致，分页使用 LIMIT。

```
SELECT id, code, datetime_dep, datetime_arr, region_dep, region_arr,
seat_first_class, seat_business_class, seat_economy_class,
price, price_business_class, price_first_class, status
FROM flight
WHERE region_dep = ? AND region_arr = ?
AND datetime_dep >= ? AND datetime_dep < ?
AND price BETWEEN ? AND ?
ORDER BY datetime_dep LIMIT ? OFFSET ?;
```

(2) 订票。实际执行时先采用第 5 章的 FOR UPDATE 锁定航班，随后根据参数选择对应舱位列。下例展示经济舱分支；商务舱和头等舱采用同样的条件更新。受影响行数为 1 才允许创建订单。

```
UPDATE flight SET seat_economy_class = seat_economy_class - 1
WHERE id = ? AND seat_economy_class > 0;
INSERT INTO orders (id_flight, id_user, id_channel, code, cabin_class, total_price, total_tax, pay_status, order_status)
VALUES (?, ?, ?, ?, 'ECONOMY', ?, ?, 'UNPAID', 'PENDING_ISSUE');
```

(3) 我的订单。当前用户标识必须参与 WHERE 条件，详情接口亦应附带该条件。

```
SELECT o.id, o.code, o.cabin_class, o.total_price, o.total_tax, o.pay_status, o.order_status,
o.create_at, f.code AS flight_code, f.datetime_dep, f.datetime_arr
FROM orders o JOIN flight f ON o.id_flight = f.id
WHERE o.id_user = ? ORDER BY o.create_at DESC;
```

(4) 退订退款。退款金额不单独落库，在响应中根据订单成交总价与当前航班退票费动态计算；随后回补原舱位并更新订单。

```
SELECT o.id, GREATEST(0, o.total_price - f.cancellation_fee) AS refund
FROM orders o JOIN flight f ON o.id_flight = f.id
WHERE o.id = ? AND o.id_user = ?;
UPDATE orders SET pay_status = 'REFUNDED', order_status = 'CANCELLED', cancel_time = CURRENT_TIMESTAMP
WHERE id = ? AND id_user = ?;
```

(5) 统计功能一：热门航班销量 Top10。以已支付或已退款订单作为成交口径，并分别统计舱位销量。

```
SELECT f.code, f.datetime_dep, COUNT(o.id) AS sales,
SUM(CASE WHEN o.cabin_class = 'ECONOMY' THEN 1 ELSE 0 END) AS economy_sales,
SUM(CASE WHEN o.cabin_class = 'BUSINESS' THEN 1 ELSE 0 END) AS business_sales,
SUM(CASE WHEN o.cabin_class = 'FIRST' THEN 1 ELSE 0 END) AS first_sales
FROM orders o JOIN flight f ON o.id_flight = f.id
WHERE o.pay_status IN ('PAID', 'REFUNDED')
GROUP BY f.id, f.code, f.datetime_dep ORDER BY sales DESC LIMIT 10;
```

(6) 统计功能二：营收总览。

```
SELECT COUNT(*) AS order_count, COALESCE(SUM(total_price), 0) AS deal_amount
FROM orders WHERE pay_status IN ('PAID', 'REFUNDED');
```

(7) 统计功能三：旅客消费排行 Top10。

```
SELECT u.id, u.username, u.real_name, COUNT(o.id) AS order_count,  
SUM(o.total_price) AS consumption  
FROM orders o JOIN sys_user u ON o.id_user = u.id  
WHERE o.pay_status IN ('PAID', 'REFUNDED')  
GROUP BY u.id, u.username, u.real_name ORDER BY consumption DESC LIMIT 10;
```

6.3 演示数据规模与统计口径

V6 迁移内置演示数据，包含 42 条航班、66 条订单、12 个用户等，合计约 175 条实体记录和 374 条关联关系。统计中的成交金额口径为支付状态属于“已支付/已退款”的订单 total_price 之和；退款金额在退订响应中动态计算，因此展示统计应明确该口径，避免把退款与售出金额混为一谈。

第七章 总结

本课程设计完成了飞机票订票系统从数据库规划、需求分析、概念模型、逻辑关系模式到物理实现的闭环设计。9 张表覆盖身份、基础资源、航班、交易、轨迹和常乘机人；通过航班号与出发时间的唯一性、订单号唯一性、常乘机人组合唯一性及路线一对一约束保证关键数据质量；通过事务和 `SELECT ... FOR UPDATE` 将余票检查、扣减与订单写入组织为原子操作，满足防超卖的核心目标。数据库设计不仅服务于业务录入，也为搜索、订单追溯和三类统计提供了清晰的索引与口径。

当前设计仍有改进空间。首先，可用 Redis 缓存热点航班搜索结果并配合失效策略，减少高峰读压；其次，可将模拟支付替换为真实渠道并保留可靠回调与幂等处理；再次，可采用消息队列异步处理退款、通知和统计聚合；当历史订单规模持续增长时，可按时间分区或冷热分离。最后，若部署场景对跨表完整性提出更高要求，可在充分评估删除策略、迁移顺序与性能后，将逻辑外键逐步固化为数据库 FOREIGN KEY 约束。